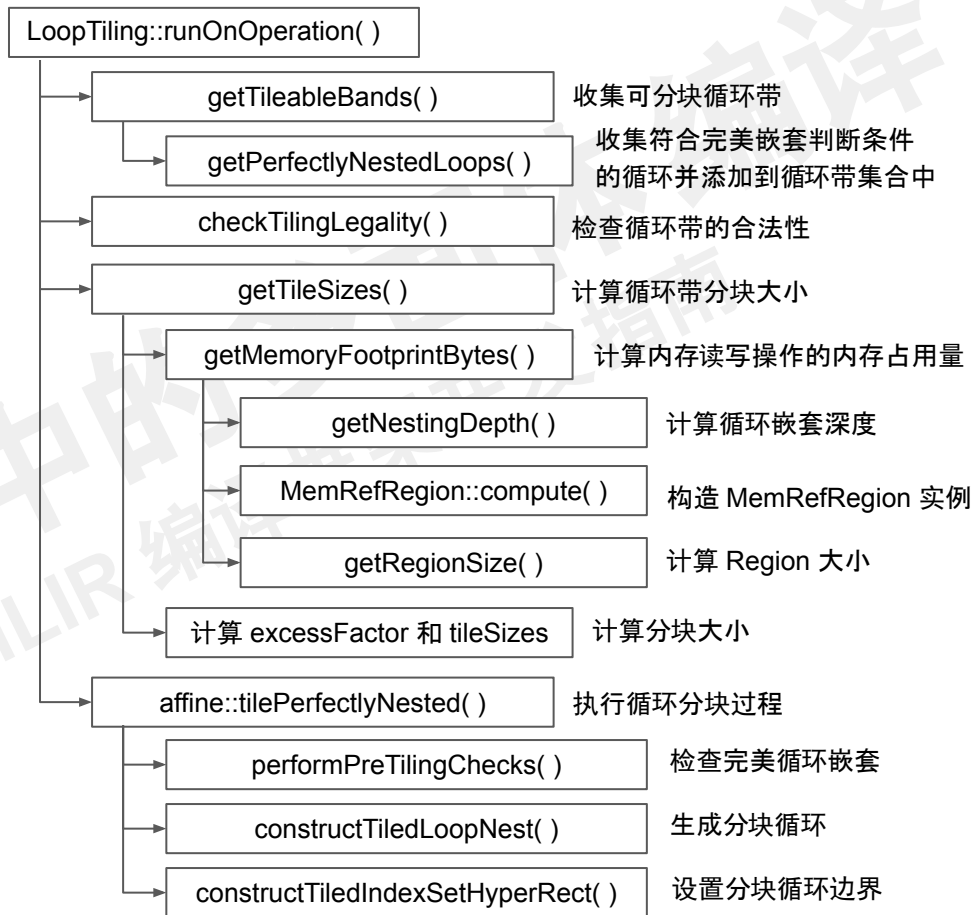


5.3.1. AffineLoopTiling pass 工作流程

LoopTiling::runOnOperation函数调用过程

主要功能：

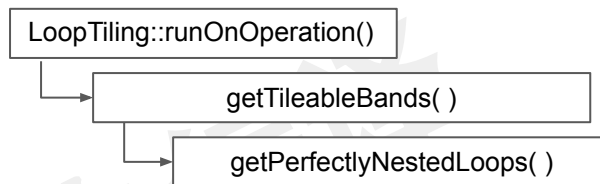
- (1) 收集可分块循环带
- (2) 计算循环带分块大小
- (3) 执行循环分块过程



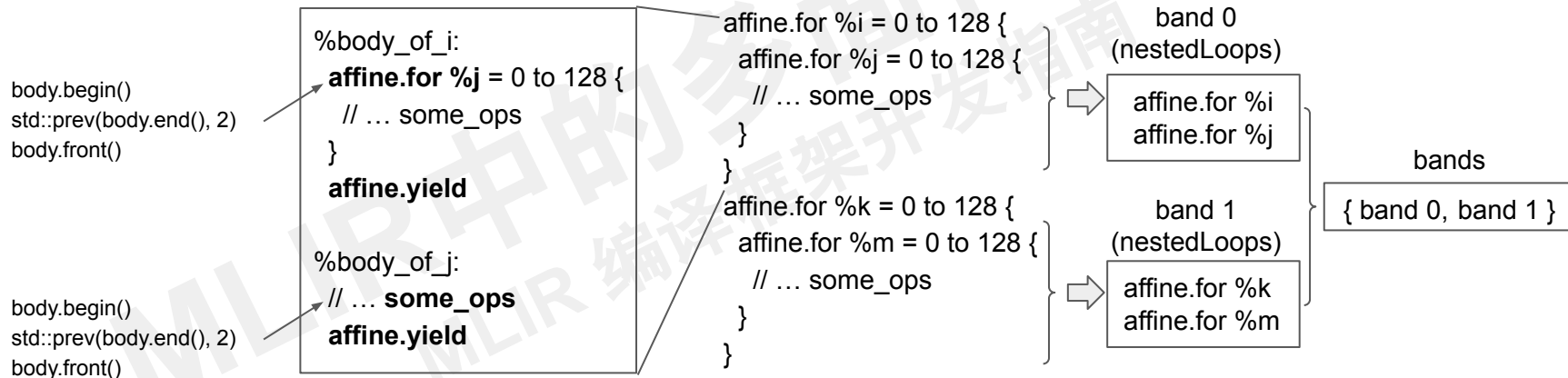
测试用例

```
func.func @simple_matmul(%arg0: memref<256x256xvector<64xf32>>, %arg1:  
memref<256x256xvector<64xf32>>, %arg2: memref<256x256xvector<64xf32>>) ->  
memref<256x256xvector<64xf32>> {  
  affine.for %i = 0 to 256 {  
    affine.for %j = 0 to 256 {  
      affine.for %k = 0 to 250 {  
        %l = affine.load %arg0[%i+1, %k+2] : memref<256x256xvector<64xf32>>  
        %r = affine.load %arg1[%k, %j] : memref<256x256xvector<64xf32>>  
        %o = affine.load %arg2[%i, %j] : memref<256x256xvector<64xf32>>  
        %m = arith.mulf %l, %r : vector<64xf32>  
        %a = arith.addf %o, %m : vector<64xf32>  
        affine.store %a, %arg2[%i, %j] : memref<256x256xvector<64xf32>>  
      }  
    }  
  }  
  return %arg2 : memref<256x256xvector<64xf32>>  
}
```

收集可分块循环带



由多个 affine.for 操作构成一组连续的完美循环嵌套被称为循环带(band)



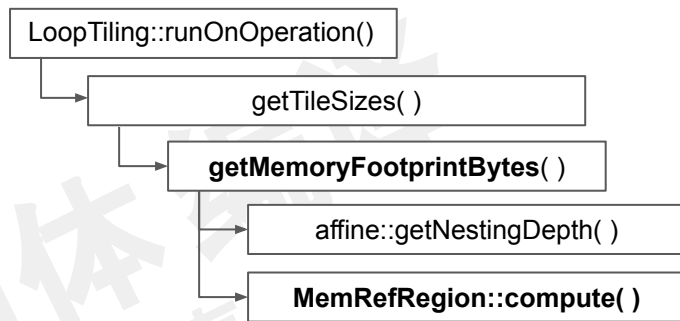
计算循环带分块大小

构造 MemRefRegion 实例

```
affine.for %i = 0 to 32 {  
  affine.for %ii = %i to (d0 + 8) (%i) {  
    affine.load %A[%ii]  
  }  
}
```

FlatAffineValueConstraints

MemRefRegion {memref = %A, write = false, {%i <= %ii <= %i + 7} }



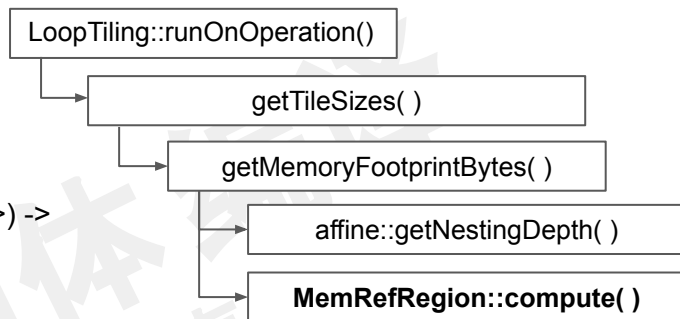
计算循环带分块大小

添加循环索引变量上下界约束

```
func.func @simple_matmul(%arg0: memref<256x256xvector<64xf32>>, %arg1:
memref<256x256xvector<64xf32>>, %arg2: memref<256x256xvector<64xf32>>) ->
memref<256x256xvector<64xf32>> {
  affine.for %i = 0 to 256 {
    affine.for %j = 0 to 256 {
      affine.for %k = 0 to 250 {
        %l = affine.load %arg0[%i+1, %k+2] : memref<256x256xvector<64xf32>>
        ...
        affine.store %a, %arg2[%i, %j] : memref<256x256xvector<64xf32>>
      }
    }
  }
  return %arg2 : memref<256x256xvector<64xf32>>
}
```

访问映射: $(d0, d1) \rightarrow (d0 + 1, d1 + 2)$

约束系统:

$$\begin{aligned} (1) * \%i + (0) * \%k + 0 &\geq 0 \\ (-1) * \%i + (0) * \%k + 255 &\geq 0 \\ (0) * \%i + (1) * \%k + 0 &\geq 0 \\ (0) * \%i + (-1) * \%k + 249 &\geq 0 \end{aligned}$$


初始状态

1. Domain: 0, Range: 2, Symbols: 0, Locals: 0
MemRefRegion {memref = %arg0, write = false, {}}

调用两次 **addAffineForOpDomain()**

2. Domain: 0, Range: **2**, Symbols: 0, Locals: 0
MemRefRegion {memref = %arg0, write = false,
{4 constraints}}

(Value Value const)

1	0	0	≥ 0
-1	0	255	≥ 0
0	1	0	≥ 0
0	-1	249	≥ 0

inequalities

计算循环带分块大小

合并访问映射

```
func.func @simple_matmul(%arg0: memref<256x256xvector<64xf32>>, %arg1:
memref<256x256xvector<64xf32>>, %arg2: memref<256x256xvector<64xf32>>) ->
memref<256x256xvector<64xf32>> {
  affine.for %i = 0 to 256 {
    affine.for %j = 0 to 256 {
      affine.for %k = 0 to 250 {
        %l = affine.load %arg0[%i+1, %k+2] : memref<256x256xvector<64xf32>>
        ...
      }
    }
  }
```

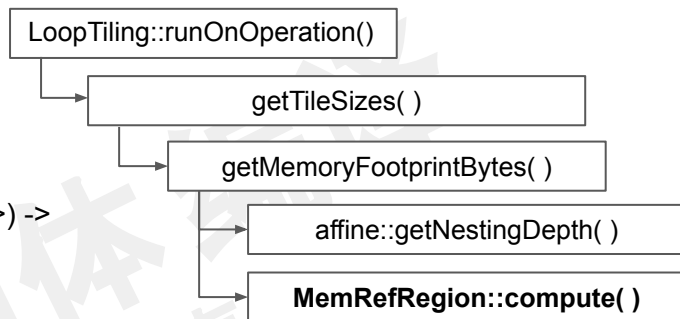
访问映射

$(d0, d1) \rightarrow (d0 + 1, d1 + 2) \Rightarrow x = \%i + 1, y = \%k + 2$

内存访问范围约束

约束系统:

```
( 1 ) * x + ( 0 ) * y + ( -1 ) * %i + ( 0 ) * %k + ( -1 ) = 0
( 0 ) * x + ( 1 ) * y + ( 0 ) * %i + ( -1 ) * %k + ( -2 ) = 0
( 0 ) * x + ( 0 ) * y + ( 1 ) * %i + ( 0 ) * %k + 0 >= 0
( 0 ) * x + ( 0 ) * y + ( -1 ) * %i + ( 0 ) * %k + 255 >= 0
( 0 ) * x + ( 0 ) * y + ( 0 ) * %i + ( 1 ) * %k + 0 >= 0
( 0 ) * x + ( 0 ) * y + ( 0 ) * %i + ( -1 ) * %k + 249 >= 0
```



调用 `composeMap( )`

3. Domain: 0, Range: 4, Symbols: 0, Locals: 0
MemRefRegion {memref = %arg0, write = false,
{6 constraints}}

(None None Value Value const)

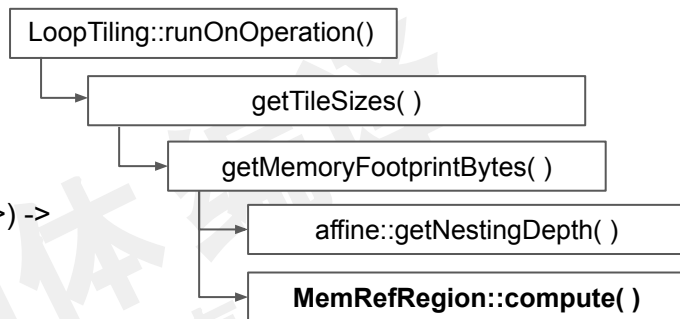
1	0	-1	0	-1	= 0
0	1	0	-1	-2	= 0
0	0	1	0	0	>= 0
0	0	-1	0	255	>= 0
0	0	0	1	0	>= 0
0	0	0	-1	249	>= 0

计算循环带分块大小

分割维度变量和符号变量

```
func.func @simple_matmul(%arg0: memref<256x256xvector<64xf32>>, %arg1:
memref<256x256xvector<64xf32>>, %arg2: memref<256x256xvector<64xf32>>) ->
memref<256x256xvector<64xf32>> {
  affine.for %i = 0 to 256 {
    affine.for %j = 0 to 256 {
      affine.for %k = 0 to 250 {
        %l = affine.load %arg0[%i+1, %k+2] : memref<256x256xvector<64xf32>>
        ...
      }
    }
  }
```

约束系统:

$$\begin{aligned}(1) * x + (0) * y + (-1) * \%i + (0) * \%k + (-1) &= 0 \\(0) * x + (1) * y + (0) * \%i + (-1) * \%k + (-2) &= 0 \\(0) * x + (0) * y + (1) * \%i + (0) * \%k + 0 &\geq 0 \\(0) * x + (0) * y + (-1) * \%i + (0) * \%k + 255 &\geq 0 \\(0) * x + (0) * y + (0) * \%i + (1) * \%k + 0 &\geq 0 \\(0) * x + (0) * y + (0) * \%i + (-1) * \%k + 249 &\geq 0\end{aligned}$$


调用 `setDimSymbolSeparation()`

4. Domain: 0, Range: **2**, Symbols: **2**, Locals: 0
MemRefRegion {memref = **%arg0**, write = false,
{6 constraints}}

(None None Value Value const)

1	0	-1	0	-1	= 0
0	1	0	-1	-2	= 0
0	0	1	0	0	>= 0
0	0	-1	0	255	>= 0
0	0	0	1	0	>= 0
0	0	0	-1	249	>= 0

计算循环带分块大小

消除 %i

```
func.func @simple_matmul(%arg0: memref<256x256xvector<64xf32>>, %arg1:  
memref<256x256xvector<64xf32>>, %arg2: memref<256x256xvector<64xf32>>) ->  
memref<256x256xvector<64xf32>> {
```

```
  affine.for %i = 0 to 256 {
```

```
    affine.for %j = 0 to 256 {
```

```
      affine.for %k = 0 to 250 {
```

```
        %l = affine.load %arg0[%i+1, %k+2] : memref<256x256xvector<64xf32>>
```

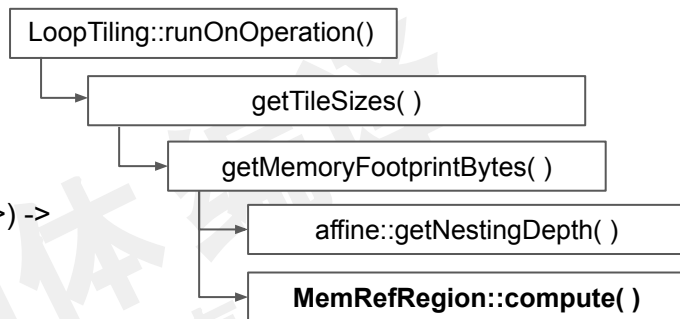
```
        ...  
      }  
      ( 1 ) * x + ( 0 ) * y + ( -1 ) * %i + ( 0 ) * %k + ( -1 ) = 0  
      ( 0 ) * x + ( 1 ) * y + ( 0 ) * %i + ( -1 ) * %k + ( -2 ) = 0  
      ( 0 ) * x + ( 0 ) * y + ( 1 ) * %i + ( 0 ) * %k + 0    >= 0  
      ( 0 ) * x + ( 0 ) * y + ( -1 ) * %i + ( 0 ) * %k + 255 >= 0  
      ( 0 ) * x + ( 0 ) * y + ( 0 ) * %i + ( 1 ) * %k + 0    >= 0  
      ( 0 ) * x + ( 0 ) * y + ( 0 ) * %i + ( -1 ) * %k + 249 >= 0
```



代入 %i = x - 1

约束系统:

```
( 0 ) * x + ( 1 ) * y + ( -1 ) * %k + ( -2 ) = 0  
( 1 ) * x + ( 0 ) * y + ( 0 ) * %k + ( -1 ) >= 0  
( -1 ) * x + ( 0 ) * y + ( 0 ) * %k + 256 >= 0  
( 0 ) * x + ( 0 ) * y + ( 1 ) * %k + 0    >= 0  
( 0 ) * x + ( 0 ) * y + ( -1 ) * %k + 249 >= 0
```



调用 cst.projectOut(%i)

5. Domain: 0, Range: **2**, Symbols: **1**, Locals: 0
MemRefRegion {memref = %arg0, write = false,
{**5** constraints}}

(None None Value const)

0	1	-1	-2	= 0
1	0	0	-1	>= 0
-1	0	0	256	>= 0
0	0	1	0	>= 0
0	0	-1	249	>= 0

计算循环带分块大小

消除 %k

```
func.func @simple_matmul(%arg0: memref<256x256xvector<64xf32>>, %arg1:  
memref<256x256xvector<64xf32>>, %arg2: memref<256x256xvector<64xf32>>) ->  
memref<256x256xvector<64xf32>> {
```

```
  affine.for %i = 0 to 256 {
```

```
    affine.for %j = 0 to 256 {
```

```
      affine.for %k = 0 to 250 {
```

```
        %l = affine.load %arg0[%i+1, %k+2] : memref<256x256xvector<64xf32>>
```

```
        ...
```

```
      }
```

```
        ( 0 ) * x + ( 1 ) * y + ( -1 ) * %k + ( -2 ) = 0
```

```
        ( 1 ) * x + ( 0 ) * y + ( 0 ) * %k + ( -1 ) >= 0
```

```
        ( -1 ) * x + ( 0 ) * y + ( 0 ) * %k + 256 >= 0
```

```
        ( 0 ) * x + ( 0 ) * y + ( 1 ) * %k + 0 >= 0
```

```
        ( 0 ) * x + ( 0 ) * y + ( -1 ) * %k + 249 >= 0
```

↓ 代入 %k = y - 2

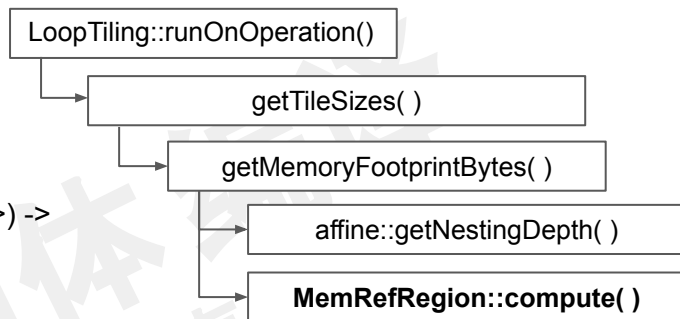
约束系统: (1) * x + (0) * y + (-1) >= 0

(-1) * x + (0) * y + 256 >= 0

(0) * x + (1) * y + (-2) >= 0

(0) * x + (-1) * y + 251 >= 0

内存访问范围为 $1 \leq x \leq 256$ 、 $2 \leq y \leq 251$



调用 cst.projectOut(%k)

6. Domain: 0, Range: **2**, Symbols: **0**, Locals: 0

MemRefRegion {memref = %arg0, write = false,
{4 constraints}}

(None None const)

1 0 -1 >= 0

-1 0 256 >= 0

0 1 -2 >= 0

0 -1 251 >= 0

计算循环带分块大小

增加内存访问索引约束

```
func.func @simple_matmul(%arg0: memref<256x256xvector<64xf32>>, %arg1:
memref<256x256xvector<64xf32>>, %arg2: memref<256x256xvector<64xf32>>) ->
memref<256x256xvector<64xf32>> {
  affine.for %i = 0 to 256 {
    affine.for %j = 0 to 256 {
      affine.for %k = 0 to 250 {
        %l = affine.load %arg0[%i+1, %k+2] : memref<256x256xvector<64xf32>>
        ...
      }
    }
  }
```

$x \geq 0, x \leq 255, y \geq 0, y \leq 255$

$(1) * x + (0) * y + (-1) \geq 0$

$(-1) * x + (0) * y + 256 \geq 0$

$(0) * x + (1) * y + (-2) \geq 0$

$(0) * x + (-1) * y + 251 \geq 0$

$(1) * x + (0) * y + 0 \geq 0$

$(-1) * x + (0) * y + 255 \geq 0$

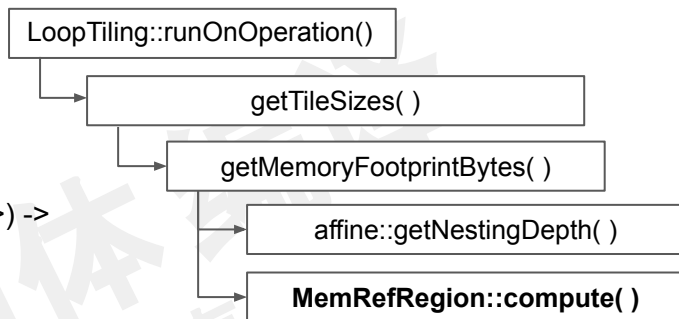
$(0) * x + (1) * y + 0 \geq 0$

$(0) * x + (-1) * y + 255 \geq 0$

约束系统:

$1 \leq x \leq 256, 2 \leq y \leq 251$

$0 \leq x \leq 255, 0 \leq y \leq 255$



调用 IntegerRelation::addBound()

7. Domain: 0, Range: **2**, Symbols: **0**, Locals: 0

MemRefRegion {memref = **%arg0**, write = false,
{8 constraints}}

(None None const)

1 0 -1 ≥ 0

-1 0 256 ≥ 0

0 1 -2 ≥ 0

0 -1 251 ≥ 0

1 0 0 ≥ 0

-1 0 255 ≥ 0

0 1 0 ≥ 0

0 -1 255 ≥ 0

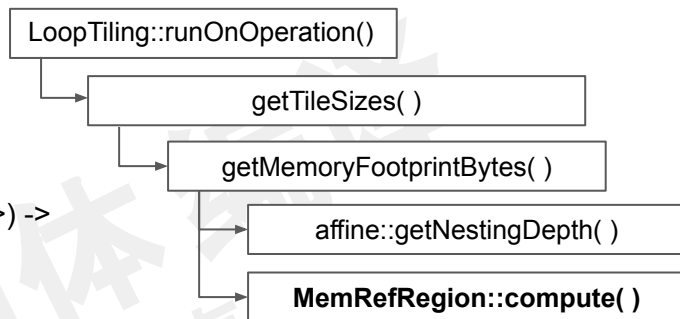
计算循环带分块大小

删除冗余约束

```
func.func @simple_matmul(%arg0: memref<256x256xvector<64xf32>>, %arg1:
memref<256x256xvector<64xf32>>, %arg2: memref<256x256xvector<64xf32>>) ->
memref<256x256xvector<64xf32>> {
  affine.for %i = 0 to 256 {
    affine.for %j = 0 to 256 {
      affine.for %k = 0 to 250 {
        %l = affine.load %arg0[%i+1, %k+2] : memref<256x256xvector<64xf32>>
        ...
      }
    }
  }
```

约束系统: $(1) * x + (0) * y + (-1) \geq 0$
 $(0) * x + (1) * y + (-2) \geq 0$
 $(0) * x + (-1) * y + 251 \geq 0$
 $(-1) * x + (0) * y + 255 \geq 0$

$1 \leq x \leq 255, 2 \leq y \leq 251$



调用 IntegerRelation::removeTrivialRedundancy()

8. Domain: 0, Range: **2**, Symbols: **0**, Locals: 0

MemRefRegion {memref = **%arg0**, write = false,
{**4** constraints}}

(None None const)

1 0 -1 ≥ 0

0 1 -2 ≥ 0

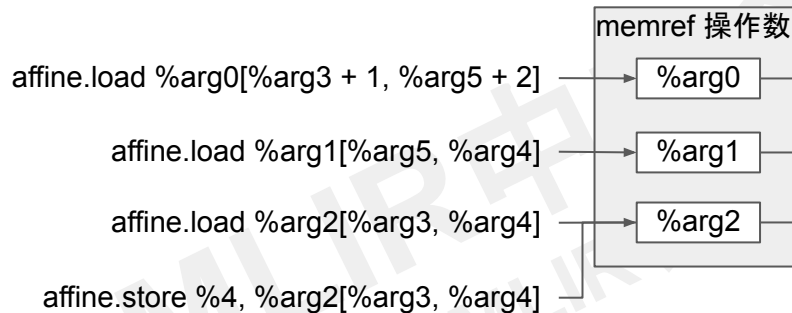
0 -1 251 ≥ 0

-1 0 255 ≥ 0

计算循环带分块大小

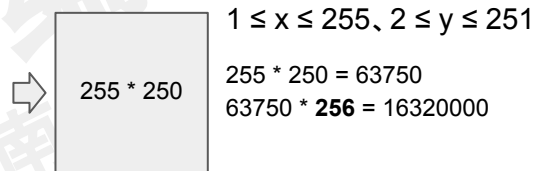
构造 regions 集合

regions 集合

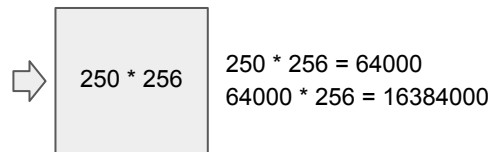


MemRefRegion 实例				
memref = %arg0				
write = false				
cts =				
1	0	-1	>= 0	
0	1	-2	>= 0	
0	-1	251	>= 0	
-1	0	255	>= 0	

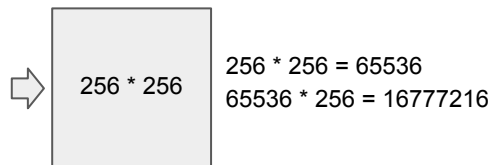
memref<256x256xvector<64xf32>



memref = %arg1				
write = false				
cts =				
1	0	0	>= 0	
-1	0	249	>= 0	
0	1	0	>= 0	
0	-1	255	>= 0	

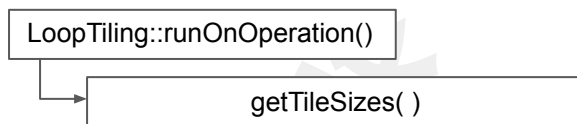


memref = %arg2			
write = false			
cts =			
1	0	0	>= 0
-1	0	255	>= 0
0	1	0	>= 0
0	-1	255	>= 0



计算循环带分块大小

计算过剩因子和分块大小



$$\text{Excess Factor} = \frac{\text{memoryFootprintBytes}}{\text{cacheSizeInKiB}}$$

$$tSize = \lfloor excessFactor^{\frac{1}{band.size()}} \rfloor$$

footprint = 49481216B, cacheSizeInKiB = 512KB

excessFactor = footprint / cacheSizeBytes = 49481216 / 524288 ≈ 95

band.size() = 3

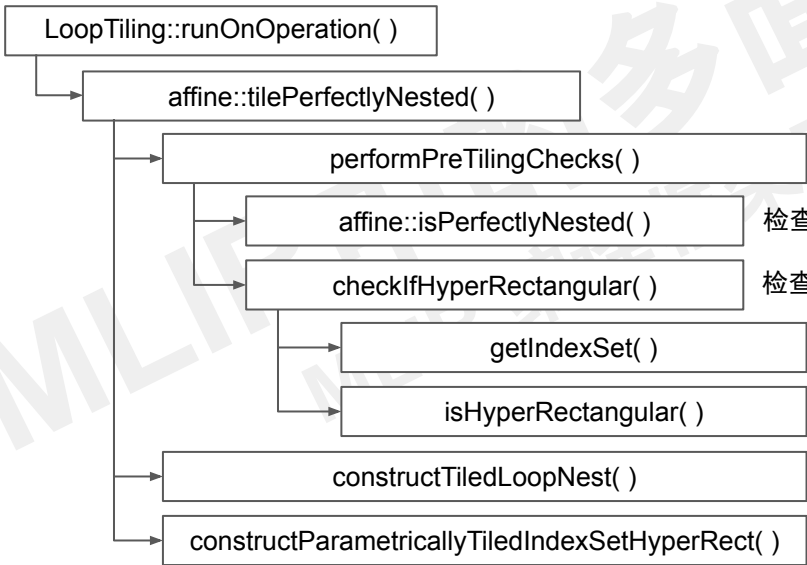
tileSizes[0] = $\lfloor 95^{\frac{1}{3}} \rfloor = \lfloor 4.56 \rfloor = 4$, cumulProductOfTileSizes = 4;

tileSizes[1] = $\lfloor 95^{\frac{1}{3}} \rfloor = \lfloor 4.56 \rfloor = 4$, cumulProductOfTileSizes = 16;

tileSizes[2] = excessFactor / cumulProductOfTileSizes ≈ 5, cumulProductOfTileSizes = 80;

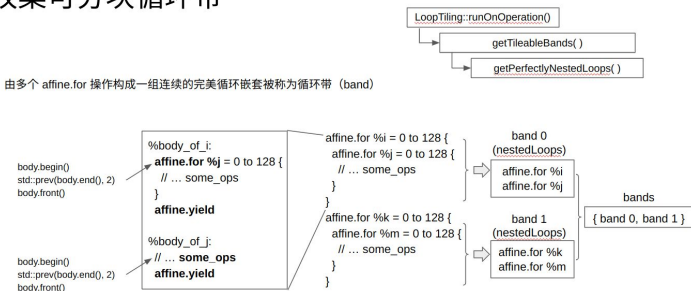
循环分块过程

- 1) 检查循环嵌套是否为完美循环嵌套和超矩形
- 2) 构造分块间循环和分块内循环
- 3) 设置分块间循环和分块内循环上下界和步长



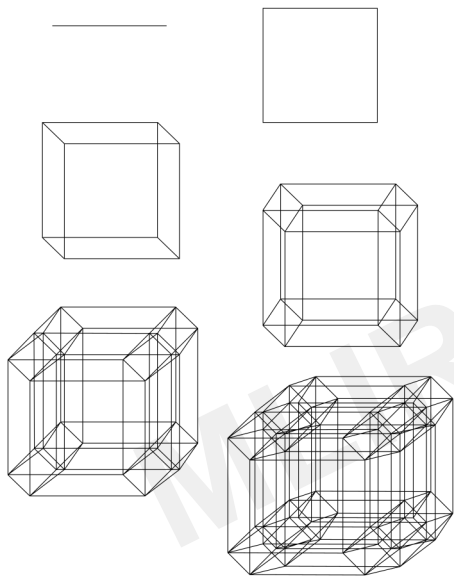
收集可分块循环带

由多个 affine.for 操作构成一组连续的完美循环嵌套被称为循环带 (band)



循环分块过程

检查循环嵌套是否为超矩形



超矩形

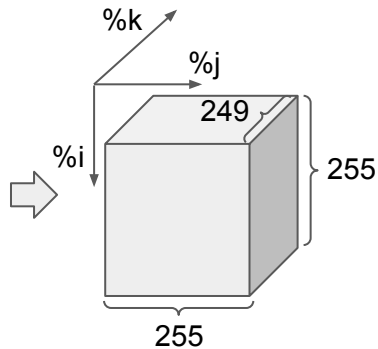
```
affine.for %i = 0 to 256 {  
  affine.for %j = 0 to 256 {  
    affine.for %k = 0 to 250 {  
      ...  
    }  
  }  
}
```

循环嵌套

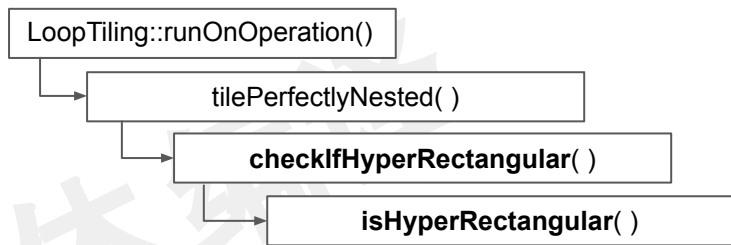


%i	%j	%k	
1	0	0	0 >= 0
-1	0	0	255 >= 0
0	1	0	0 >= 0
0	-1	0	255 >= 0
0	0	1	0 >= 0
0	0	-1	249 >= 0

约束系统



超矩形



循环分块过程

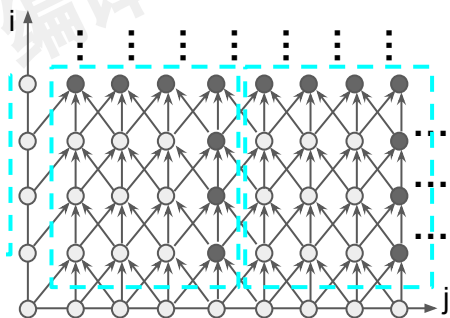
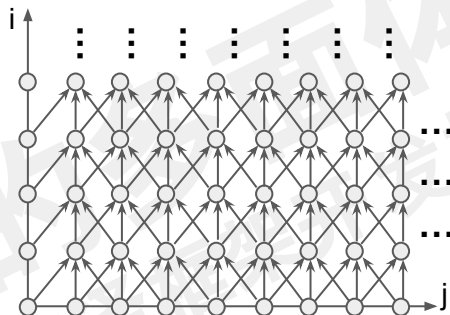
分块间循环与分块内循环

```
for (int i = 1; i <= 20; i++) {  
  for (int j = 1; j <= 20; j++) {  
    A[i][j] = A[i-1][j-1] + A[i-1][j] + A[i-1][j+1];  
  }  
}
```

原始循环嵌套

```
for (int i = 1; i <= 20; i += 4) { // 分块间循环  
  for (int j = 1; j <= 20; j += 4) { // 分块间循环  
    for (int m = i; m < i+4; m++) { // 分块内循环  
      for (int n = j; n < j+4; n++) { // 分块内循环  
        A[m][n] = A[m-1][n-1] + A[m-1][n] + A[m-1][n+1];  
      }  
    }  
  }  
}
```

分块循环嵌套



LoopTiling::runOnOperation()

tilePerfectlyNested()

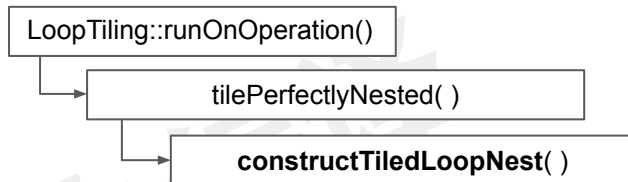
constructTiledLoopNest()

$$\text{Excess Factor} = \frac{\text{memoryFootprintBytes}}{\text{cacheSizeInKiB}}$$

$$tSize = \lfloor excessFactor^{\frac{1}{\text{band.size}()}} \rfloor$$

循环分块过程

构造分块内循环



```
affine.for %i = 0 to 256 {  
  affine.for %j = 0 to 256 {  
    affine.for %k = 0 to 250 {  
      %l = affine.load %arg0[%i+1, %k+2] ...  
      ...  
    }  
  }  
  ...  
}
```

1

```
affine.for %arg3 = 0 to 0 {  
  affine.for %arg4 = 0 to 256 {  
    affine.for %arg5 = 0 to 256 {  
      affine.for %arg6 = 0 to 250 {  
        %l = affine.load %arg0[%i+1, %k+2] ...  
        ...  
      }  
    }  
    ...  
  }  
  ...  
}
```

2

```
affine.for %arg3 = 0 to 0 {  
  affine.for %arg4 = 0 to 0 {  
    affine.for %arg5 = 0 to 256 {  
      affine.for %arg6 = 0 to 256 {  
        affine.for %arg7 = 0 to 250 {  
          %l = affine.load %arg0[%i+1, %k+2] ...  
          ...  
        }  
      }  
      ...  
    }  
    ...  
  }  
  ...  
}
```

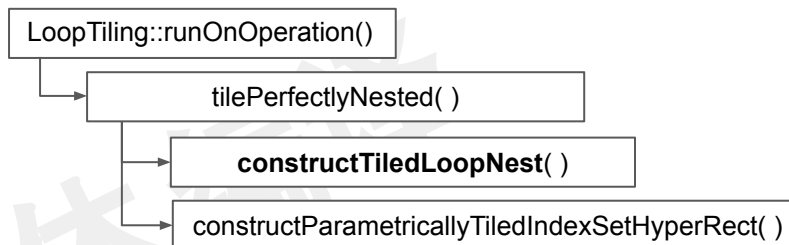
3

```
affine.for %arg3 = 0 to 0 {  
  affine.for %arg4 = 0 to 0 { }分块内循环  
    affine.for %arg5 = 0 to 0 {  
      affine.for %arg6 = 0 to 256 {  
        affine.for %arg7 = 0 to 256 { }原始循环  
          affine.for %arg8 = 0 to 250 {  
            %l = affine.load %arg0[%i+1, %k+2] ...  
            ...  
          }  
        }  
        ...  
      }  
      ...  
    }  
    ...  
  }  
  ...  
}
```

4

循环分块过程

构造分块间循环



```
affine.for %arg3 = 0 to 0 {  
  affine.for %arg4 = 0 to 0 {  
    affine.for %arg5 = 0 to 0 {  
      affine.for %arg6 = 0 to 0 {  
        affine.for %arg7 = 0 to 0 {  
          affine.for %arg8 = 0 to 0 {  
            %0 = affine.load %arg0[%arg9 + 1, %arg11 + 2] : ...  
            ...  
            affine.for %arg9 = 0 to 256 {  
              affine.for %arg10 = 0 to 256 {  
                affine.for %arg11 = 0 to 250 {  
                  ...  
                }  
              }  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

分块间循环

分块内循环

原始循环

1

```
affine.for %arg3 = 0 to 256 step 4 {  
  affine.for %arg4 = 0 to 256 step 4 {  
    affine.for %arg5 = 0 to 250 step 5 {  
      affine.for %arg6 = %arg3 to %arg3+4 {  
        affine.for %arg7 = %arg4 to %arg4+4 {  
          affine.for %arg8 = %arg5 to %arg5+5 {  
            %0 = affine.load %arg0[%arg9 + 1, %arg11 + 2] : ...  
            ...  
          }  
        }  
      }  
    }  
  }  
}
```

分块间循环

分块内循环

2

$$\text{Excess Factor} = \frac{\text{memoryFootprintBytes}}{\text{cacheSizeInKiB}}$$

$$tSize = \lfloor excessFactor^{\frac{1}{band.size()}} \rfloor$$